

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«ВЛАДИВОСТОКСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «ВВГУ»)
Институт информационных технологий и анализа данных
Кафедра информационных технологий и систем

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ

по дисциплине «Информатика и основы программирования»

Проверка условий

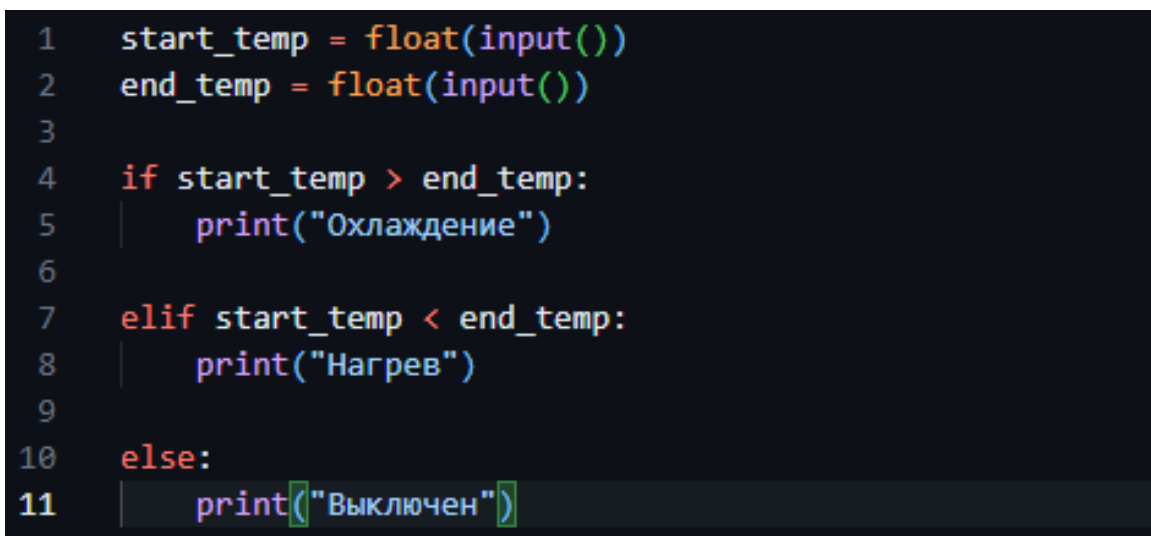
Студент		
гр. БИН-26-1	_____	У. Т. Чан-Ван-Ю
Преподаватель	_____	М. В. Водяницкий

1 Выполнение работы

1.1 Задание 1. Управление кондиционером

Первое задание моделирует работу системы управления климатом. Пользователь вводит два вещественных числа, представляющих текущую и желаемую температуры воздуха, каждое на отдельной строке. В зависимости от соотношения этих значений программа должна определить режим работы кондиционера: Охлаждение, если текущая температура превышает желаемую; Нагрев, если она ниже желаемой; и Выключен, если температуры совпадают.

Исходный код программы с результатами её выполнения приведён на рисунке 1.



```
1  start_temp = float(input())
2  end_temp = float(input())
3
4  if start_temp > end_temp:
5      print("Охлаждение")
6
7  elif start_temp < end_temp:
8      print("Нагрев")
9
10 else:
11     print("Выключен")
```

Рисунок 1 – Исходный код и результат выполнения первого задания

Для реализации алгоритма используется конструкция if-elif-else, позволяющая выбрать один из трёх взаимоисключающих вариантов. Ввод данных осуществляется функцией `input()` с последующим преобразованием в вещественный тип посредством `float()`. Сравнение температур производится операторами `>`, `<` и `==`, после чего соответствующий результат выводится на экран функцией `print()`.

1.2 Задание 2. Расчёт стоимости билета

Во втором задании требуется рассчитать стоимость билета в зависимости от возраста пассажира. Пользователь вводит базовую стоимость (вещественное число) и возраст (целое число) на отдельных строках. Программа должна проверить корректность входных данных: стоимость обязана быть положительной, а возраст — находиться в диапазоне от 0 до 120 лет включительно. При нарушении любого из этих условий выводится сообщение `Ошибка`. Для корректных данных цена рассчитывается согласно возрастной категории: дети до 5 лет — бесплатно, от 6 до 17 лет — 50% от базовой стоимости, от 18 до 59 лет — полная стоимость, от 60 до 120 лет — 70% от базовой стоимости.

Исходный код программы с результатами её выполнения приведён на рисунке 2.

```

1  ceil = float(input())
2  age = int(input())
3
4  if age > 120 or age < 0 or ceil < 0:
5      print("Ошибка")
6
7  elif 0 <= age <= 5:
8      print(f"Стоимость: {int(ceil * 0):.2f}")
9
10 elif 6 <= age <= 17:
11     print(f"Стоимость: {int((ceil / 100) * 50):.2f}")
12
13 elif 18 <= age <= 59:
14     print(f"Стоимость {int(ceil):.2f}")
15
16 elif 60 <= age <= 120:
17     print(f"Стоимость {int((ceil/100) * 70):.2f}")

```

Рисунок 2 – Исходный код и результат выполнения второго задания

Проверка ограничений реализована в первом условном блоке `if`, объединяющем три условия через логический оператор `or`. Далее следует цепочка `elif`, в которой каждая ветвь соответствует своему возрастному диапазону. Для вычисления доли базовой стоимости применяются арифметические операции, а итоговый результат форматируется с двумя знаками после десятичной точки при помощи спецификатора `.2f` внутри `f`-строки.

1.3 Задание 3. Определение положения точки

Третье задание заключается в определении положения точки на координатной плоскости по её декартовым координатам `x` и `y`, вводимым пользователем как два вещественных числа. Программа должна распознать один из семи возможных случаев: начало координат (обе координаты равны нулю), принадлежность оси `X` или оси `Y`, а также нахождение в одной из четырёх четвертей плоскости.

Исходный код программы с результатами её выполнения приведён на рисунке 3.

```
1  x = float(input())
2  y = float(input())
3
4  if x == 0 and y == 0:
5      print("Начало координат")
6
7  elif y == 0 and x != 0:
8      print("Ось X")
9
10 elif x == 0 and y != 0:
11     print("Ось Y")
12
13 elif x > 0 and y > 0:
14     print("I четверть")
15
16 elif x < 0 and y > 0:
17     print("II четверть")
18
19 elif x < 0 and y < 0:
20     print("III четверть")
21
22 elif x > 0 and y < 0:
23     print("IV четверть")
```

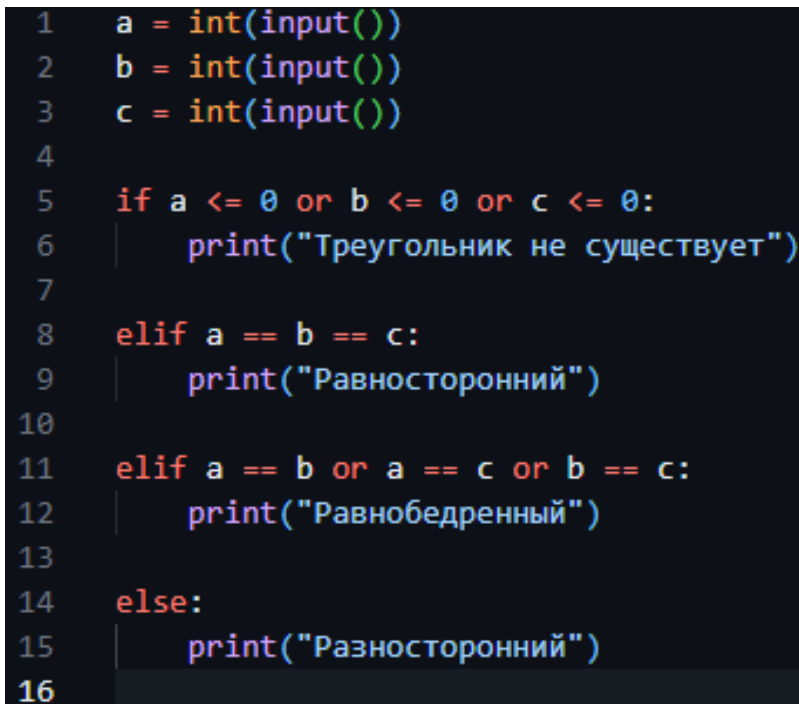
Рисунок 3 – Исходный код и результат выполнения третьего задания

Алгоритм построен на последовательной проверке условий с использованием конструкции if-elif-else. Сначала обрабатываются особые случаи (равенство координат нулю), затем — принадлежность осям, и в завершение — определение четверти по знакам координат. Сравнения выполняются операторами ==, !=, > и <, а для объединения условий применяется логический оператор and.

1.4 Задание 4. Проверка треугольника

Четвёртое задание требует определить тип треугольника по длинам его трёх сторон, вводимых пользователем как целые числа. Сначала программа проверяет существование треугольника: если хотя бы одна сторона неположительна или сумма каких-либо двух сторон не превышает третью, выводится сообщение Треугольник не существует. Для существующего треугольника определяется его вид: Равносторонний (все три стороны равны), Равнобедренный (равны ровно две стороны) или Разносторонний (все стороны различны).

Исходный код программы с результатами её выполнения приведён на рисунке 4.

A screenshot of a code editor with a dark background. It shows a Python script for checking the existence and type of a triangle based on three side lengths (a, b, c). The script uses input() to get the side lengths and print() to output the result. It uses an if-elif-else structure to handle different cases: invalid triangle, equilateral, isosceles, and scalene.

```
1  a = int(input())
2  b = int(input())
3  c = int(input())
4
5  if a <= 0 or b <= 0 or c <= 0:
6      print("Треугольник не существует")
7
8  elif a == b == c:
9      print("Равносторонний")
10
11 elif a == b or a == c or b == c:
12     print("Равнобедренный")
13
14 else:
15     print("Разносторонний")
16
```

Рисунок 4 – Исходный код и результат выполнения четвёртого задания

Проверка существования треугольника реализована через условие, объединяющее проверку на положительность сторон и неравенство треугольника. Для определения типа используется цепочка if-elif-else: сначала проверяется равенство всех трёх сторон, затем — наличие хотя бы одной пары равных сторон, и в противном случае треугольник признаётся разносторонним.

1.5 Задание 5. Определение високосного года

В пятом задании необходимо определить, является ли введённый пользователем год високосным. Номер года — целое число; если оно выходит за пределы диапазона от 1 до 9999, программа выводит Ошибка. Для корректного значения применяется правило: год считается високосным, если он делится на 400, либо одновременно делится на 4 и не делится на 100. В противном случае год признаётся невисокосным.

Исходный код программы с результатами её выполнения приведён на рисунке 5.

```
1 year = int(input())
2
3 if year < 1 or year > 9999:
4     print("Ошибка")
5
6 if ((year % 400) == 0) or (((year % 4) == 0) and ((year % 100) != 0)):
7     print("Високосный")
8
9 else:
10    print("Невисокосный")
```

Рисунок 5 – Исходный код и результат выполнения пятого задания

Сначала выполняется проверка допустимого диапазона года. Далее применяется составное логическое условие с использованием операторов остатка от деления (%), а также логических операторов `or` и `and` для реализации правила определения високосного года. Результат выводится функцией `print()` в виде соответствующей строки.

2 Вывод

В ходе выполнения лабораторной работы были изучены конструкции условного программирования в языке Python. На практике освоены использование операторов сравнения (`==`, `!=`, `>`, `<`, `>=`, `<=`), логических операторов (`and`, `or`, `not`), а также построение цепочек `if-elif-else` для выбора одного из нескольких взаимоисключающих вариантов выполнения программы. Полученные навыки позволяют реализовывать алгоритмы, поведение которых зависит от входных данных, что является фундаментальной основой для разработки более сложных программных систем.